

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 2 — Progettare il prompt

Ragionare, sui modelli di oggi

«Let’s think step by step»

Il *chain-of-thought*, nel 2022: una frase in più nel prompt

- ***Chain-of-thought***, catena di pensiero: far **ragionare il modello per passi**, scritti, prima della risposta — nel 2022 uno studio (Wei e altri) lo otteneva con alcuni esempi risolti passo per passo, nel prompt
- Un altro studio del 2022 (Kojima e altri): **nessun esempio**, una frase sola prima della risposta, «Let’s think step by step» — su un modello di allora, text-davinci-002, l’accuratezza sui problemi aritmetici di MultiArith passa dal **17,7 al 78,7 %**
- Ma solo sui modelli **grandi**: secondo Wei e altri i guadagni arrivano verso i 100 miliardi di parametri; sotto, catene di passi fluenti ma illogiche

Allora il modello ragionava scrivendo: **lo spazio per ragionare era la risposta**.

Il problema degli sportelli

M2-D05, dal vivo: lo stesso problema, di getto e passo passo, sui tre modelli

Indovinate, a voce: il problema degli sportelli, risposta **di getto** — quale dei tre modelli **sbaglia**?

sportelli.md, poi di-getto.txt

```
In un ufficio ci sono 3 sportelli. Ogni sportello serve una
persona ogni 4 minuti. Alle 9:00 ci sono 27 persone in fila. A
che ora esce l'ultima persona?
Rispondi solo con l'orario, senza spiegazioni.
```

Passo passo: l'ultima riga diventa «Ragiona passo passo, poi scrivi la risposta finale.»

```
claude -p "$(cat sportelli.md)"$'\n'"$(cat di-getto.txt)" \
  --model haiku --effort low --tools "" \
  --system-prompt "$(cat istruzione.txt)" --strict-mcp-config \
  --output-format json
# ... le altre otto chiamate e la tabella: README, passo 2
```

Demo **M2-D05**, dal vivo: guardate l'**orario** di ogni risposta e i **token in uscita**

Nel repository delle esercitazioni:
`m2_prompting_avanzato/s1_progettare_il_prompt/m2_6_ragionamento/M2-D05_ragionare_oggi/README.md`, passo 2

Il problema degli sportelli

M2-D05, dal vivo: lo stesso problema, di getto e passo passo, sui tre modelli



Il problema degli sportelli a Haiku, Sonnet e Opus, di getto e passo passo: sei chiamate a effort low, nessuno strumento; i token sono quelli in uscita

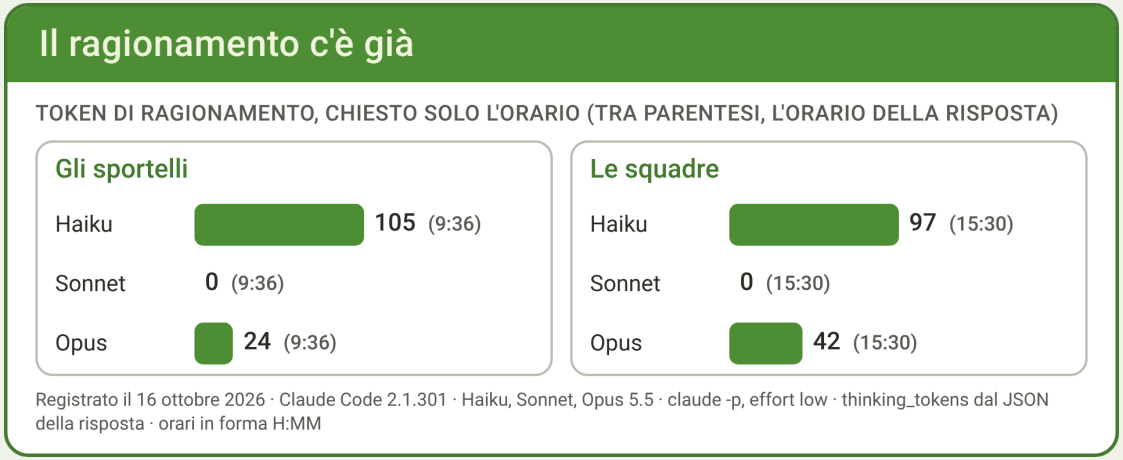
Tutte e sei le risposte danno l'orario giusto

© Simone Scannapieco — figura dalla registrazione del 16 ottobre 2026

- **Nessuno sbaglia**: di getto e passo passo, i tre modelli danno **l'orario giusto** — passo passo cambia **ciò che vedete**, non la risposta: da 315 a 530 token in uscita, di getto da 29 a 110
- Chiesto **solo l'orario**, Haiku e Opus usano comunque **token di ragionamento**, e Sonnet scrive il calcolo prima dell'orario: il ragionamento **c'è già**, anche senza chiederlo

Il ragionamento c’è già

Sui modelli 5.5 il *thinking* non si spegne: la leva è l’effort



Chiesto solo l’orario, una chiamata per modello a effort low: i token di ragionamento e, tra parentesi, l’orario della risposta — sugli sportelli e su un problema più lungo, 26 linee da riparare con tre squadre

© Simone Scannapieco — figura dalla registrazione del 16 ottobre 2026

- Il *thinking*, il ragionamento prima della risposta, è **adattivo**: Claude decide da sé quando e quanto ragionare, secondo l’effort e la complessità della richiesta
- Sui modelli 5.5 il *thinking* **non si spegne**, né in Claude Code né in claude.ai: nella figura, chiesto solo l’orario, Haiku e Opus ragionano lo stesso
- La leva è l’**effort**: `--effort` o `/effort` in Claude Code, il menu del modello in claude.ai — più effort, più ragionamento, più token: i limiti d’uso arrivano prima
- In Claude Code, `ultrathink` nel prompt chiede più ragionamento **per quel turno**, senza cambiare l’effort; «think hard» invece è testo come un altro

Che cosa consiglia Anthropic

Sul ragionamento: quattro consigli, dalla documentazione e dalla ricerca di Anthropic

- **Istruzioni generali**, non passi prescrittivi — nella documentazione, in italiano:

> *Preferisci istruzioni generali a passaggi prescrittivi. Un prompt come "ragiona a fondo" spesso produce un ragionamento migliore di un piano passo-passo scritto a mano.*
>
> **Documentazione di Anthropic, versione italiana**

- Il ragionamento scritto si chiede **per leggerlo**: il passo passo chiesto a mano è un ripiego per quando il *thinking* è spento — e su Opus e Sonnet 5.5 la richiesta di scrivere il proprio ragionamento **può essere rifiutata**
- Una **breve spiegazione** va sempre bene: le evidenze dietro un risultato. E il ragionamento si legge anche senza chiederlo: in claude.ai la sezione «Thinking» ne mostra un riassunto, in Claude Code `Ctrl+O`
- Il ragionamento mostrato **non è una prova**: Anthropic avverte che non è certo che rappresenti ciò che accade davvero nel modello — leggetelo come un indizio, e verificate il risultato

Le strategie, in una slide

Da nessuna istruzione allo schema — sei modi di chiedere

Implicit	solo i dati, senza istruzione: il modello indovina che cosa volete — ogni ingrediente che manca è una decisione lasciata a lui
Zero-shot	istruzione e dati, nessun esempio — il default
Few-shot	dopo l'istruzione, alcuni esempi: insegnano la forma — le etichette e le regole si scrivono
Role-based	ciò che non cambia da una richiesta all'altra nel *system prompt*: ruolo, regole, formato; il caso e i dati nel messaggio
***Thinking* ed effort**	il ragionamento c'è già: si regola con l'effort, e quello scritto si chiede per leggerlo
Output strutturato	la forma della risposta imposta da uno schema — quando il risultato entra in un programma

Quale strategia per quale compito

La tabella dei criteri — anche nella scheda `riferimenti/pattern-di-prompt.md`

Compito	Strategia	Perché
Riassumere, tradurre	*Zero-shot*, con formato e vincoli	compito comune: basta dire come lo volete
Classificare, estrarre	Gli esempi e le etichette ammesse, o uno schema	gli esempi insegnano la forma; le etichette si scrivono
Calcolare, decidere	Lasciarlo ragionare, con l'effort più alto; leggere il ragionamento quando conta	il ragionamento c'è già: si regola, e si legge quando serve
Rispondere ai clienti	Ruolo, contesto, procedure, esempi di stile	il tono si mostra; che cosa si può promettere lo dicono le procedure
Alimentare un programma	Uno schema JSON: in Claude Code `--json-schema`	lo schema garantisce la forma, non il contenuto

Sui modelli di oggi il ragionamento c'è già: la leva è l'effort, e il ragionamento si chiede per leggerlo.